

Operators & Conditionals

Teaching Your Program to Make Decisions

Session 4

What We're Covering Today

- Planning before coding — pseudocode & flowcharts
- Arithmetic operators
- Floor division (//) and modulo (%)
- Comparison operators
- Logical operators — and, or, not
- Operator precedence & the BODMAS rule
- if / elif / else — making real decisions
- Nested if — a decision inside a decision
- Bonus: assignment, bitwise, identity, membership & escape sequences

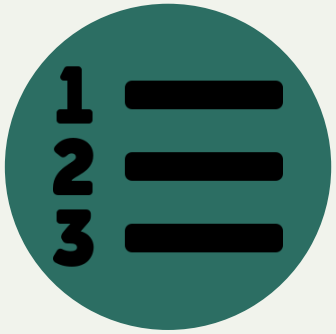


PART 1

Plan Before You Code

The best programmers write the least code first — on paper.

What Is Pseudocode?



Plain English, step by step

- Pseudocode is just your plan, written in plain English — not real code
- It has no strict rules, no syntax to get right, and nothing can "error out"
- You write it so you can think clearly, before worrying about Python's exact grammar
- Example: "IF it is raining, THEN take an umbrella, ELSE don't"

Pseudocode → Python

- START
- Ask: is it raining?
- IF yes → take an umbrella
- ELSE → don't take one
- END

```
# the exact same plan, now in real Python
is_raining = True   # try changing this to False

if is_raining:
    print("Take an umbrella")
else:
    print("Don't take one")
```

Practice Set — Planning

Try these yourself first — no code shown, that's the point

1. Write pseudocode for checking if a year is a leap year

2. Turn your own morning routine into 5 simple pseudocode steps



PART 2

Operators

Arithmetic Operators — The Basics

+ Addition

5 + 2 → 7

− Subtraction

5 - 2 → 3

* Multiplication

5 * 2 → 10

/ Division

5 / 2 → 2.5

** Exponent

5 ** 2 → 25

Floor Division (//) and Modulo (%)

- // gives you the whole-number part of a division, and throws away the rest
- % gives you only the remainder — nothing else
- Together, they split a number into "how many whole groups" and "what's left over"
- A very common use: checking even or odd — any number % 2 is 0 only when it's even

```
print(17 // 5)   # 3   -> 5 fits into 17 three whole times
print(17 % 5)    # 2   -> with 2 left over

print(10 % 2)    # 0   -> even
print(7 % 2)     # 1   -> odd
```



Arithmetic: Numbers vs. Variables

Directly on Numbers

```
print(5 + 3)   # 8
print(5 - 3)   # 2
print(5 * 3)   # 15
print(5 / 3)   # 1.666...
print(5 // 3)  # 1
print(5 % 3)   # 2
print(5 ** 3)  # 125
```

The Same Math, on Variables

```
a = 10
b = 3

print(a + b)   # 13
print(a - b)   # 7
print(a * b)   # 30
print(a / b)   # 3.333...
print(a // b)  # 3
print(a % b)   # 1
print(a ** b)  # 1000
```

Arithmetic Mini Programs (1 & 2)

1. Simple Adder

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
print("Sum:", num1 + num2)
```

2. Area of a Rectangle

```
length = float(input("Enter length: "))
width = float(input("Enter width: "))
area = length * width
print("Area:", area)
```

Arithmetic Mini Programs (3 & 4)

3. Minutes → Hours & Minutes

```
total_minutes = int(input("Enter total minutes: "))
hours = total_minutes // 60
minutes = total_minutes % 60
print(hours, "hours and", minutes, "minutes")
```

4. Average of Three Numbers

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))
average = (a + b + c) / 3
print("Average:", average)
```



Comparison Operators



== Equal to

5 == 5 → True



!= Not equal

5 != 3 → True



< Less than

3 < 5 → True



> Greater than

5 > 3 → True



<= Less or equal

5 <= 5 → True



>= Greater or equal

5 >= 6 → False

Comparison: Numbers vs. Variables

Directly on Numbers

```
print(5 == 5)   # True
print(5 != 3)   # True
print(3 < 5)    # True
print(5 > 3)    # True
print(5 <= 5)   # True
print(5 >= 6)   # False
```

The Same Checks, on Variables

```
age = 20
voting_age = 18

print(age == voting_age)  # False
print(age != voting_age)  # True
print(age < voting_age)   # False
print(age > voting_age)   # True
print(age <= voting_age)  # False
print(age >= voting_age)  # True
```

Comparison Mini Programs (1 & 2)

1. Are Two Numbers Equal?

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
print(num1 == num2)
```

2. Voting Eligibility Check

```
age = int(input("Enter your age: "))
print(age >= 18)
```

Comparison Mini Programs (3 & 4)

3. Compare Two Scores

```
score1 = int(input("Enter score 1: "))
score2 = int(input("Enter score 2: "))

print(score1 > score2)
print(score1 < score2)
```

4. Chained Range Check

```
number = int(input("Enter a number: "))

# True only if number is between 1 and 10
print(1 <= number <= 10)
```

Logical Operators



and

True only if BOTH sides are true — age >= 18 and has_id



or

True if AT LEAST ONE side is true — is_weekend or is_holiday



not

Flips True to False, and False to True — not is_raining



Logical: Direct Values vs. Variables

Directly on True / False

```
print(True and False)   # False
print(True or False)    # True
print(not True)         # False
```

On Variables

```
is_sunny = True
is_weekend = True

print(is_sunny and is_weekend)   # True
print(is_sunny or is_weekend)    # True
print(not is_sunny)              # False
```

Logical Mini Programs (1 & 2)

1. Can You Watch the Movie?

```
age = int(input("Enter your age: "))
has_ticket = True

print(age >= 13 and has_ticket)
```

2. Good Day for a Picnic?

```
is_sunny = True
is_windy = False

print(is_sunny and not is_windy)
```

Logical Mini Programs (3 & 4)

3. Simple Login Check

```
username = input("Username: ")
password = input("Password: ")

correct_username = "admin"
correct_password = "1234"

print(username == correct_username
    and password == correct_password)
```

4. Discount Eligibility

```
is_member = False
total_spent = 120

print(is_member or total_spent > 100)
```

Operator Precedence — What Runs First?



You already know this rule

- When an expression has more than one operator, Python must decide which one to calculate first
- This order is called operator precedence
- You already learned this rule in school — it's called BODMAS (or PEMDAS)
- Brackets → Orders (powers) → Division & Multiplication → Addition & Subtraction



BODMAS, Step by Step



B — Brackets

Always calculated first ( )



O — Orders

Powers / exponents **



DM — Division & Multiplication

Left to right / // * %



AS — Addition & Subtraction

Left to right + -

BODMAS in Python — Worked Examples

- Python follows this exact order automatically — you never need to tell it to
- When an expression feels unclear, add your own brackets to make the order obvious
- Try predicting each answer before reading the comment

```
print(2 + 3 * 4)        # 14, not 20 — multiplication first
print((2 + 3) * 4)      # 20 — brackets override the order
print(2 ** 3 + 1)       # 9 — exponent before addition
print(10 - 2 * 3 // 2)  # 7  ->  2*3=6, 6//2=3, 10-3=7
```

The Full Order (Arithmetic, Comparison & Logic)



1. Brackets

( )



2. Exponent

**



3. × ÷

* / // %



4. + −

+ -



5. Comparisons

== != < > <= >=



6. not, and, or

in that order

Practice Set — Operators

Try these yourself first — no code shown, that's the point

1. Print the result of 17 // 5 and 17 % 5, without looking back at the earlier slide
2. Store two numbers in variables and print their sum, difference, and product
3. Write a comparison expression checking if 25 is greater than 30
4. Use and to check if a number is between 10 and 20
5. Use not to flip a True value to False, and print the result
6. Predict the output of 2 ** 3 // 2 on paper before you run it

Let students actually try these before revealing any answers.



PART 3

Making Decisions With if

What Is a Flowchart?



A picture of your plan

- A flowchart is the same plan as pseudocode, but drawn as a picture
- Each shape means something specific — you'll only ever need three of them
- Following the arrows shows you exactly what your program will do, and in what order
- Great for decisions especially — you can literally see every path your code could take

The 3 Shapes You Need



Oval

Start or End of the whole process



Rectangle

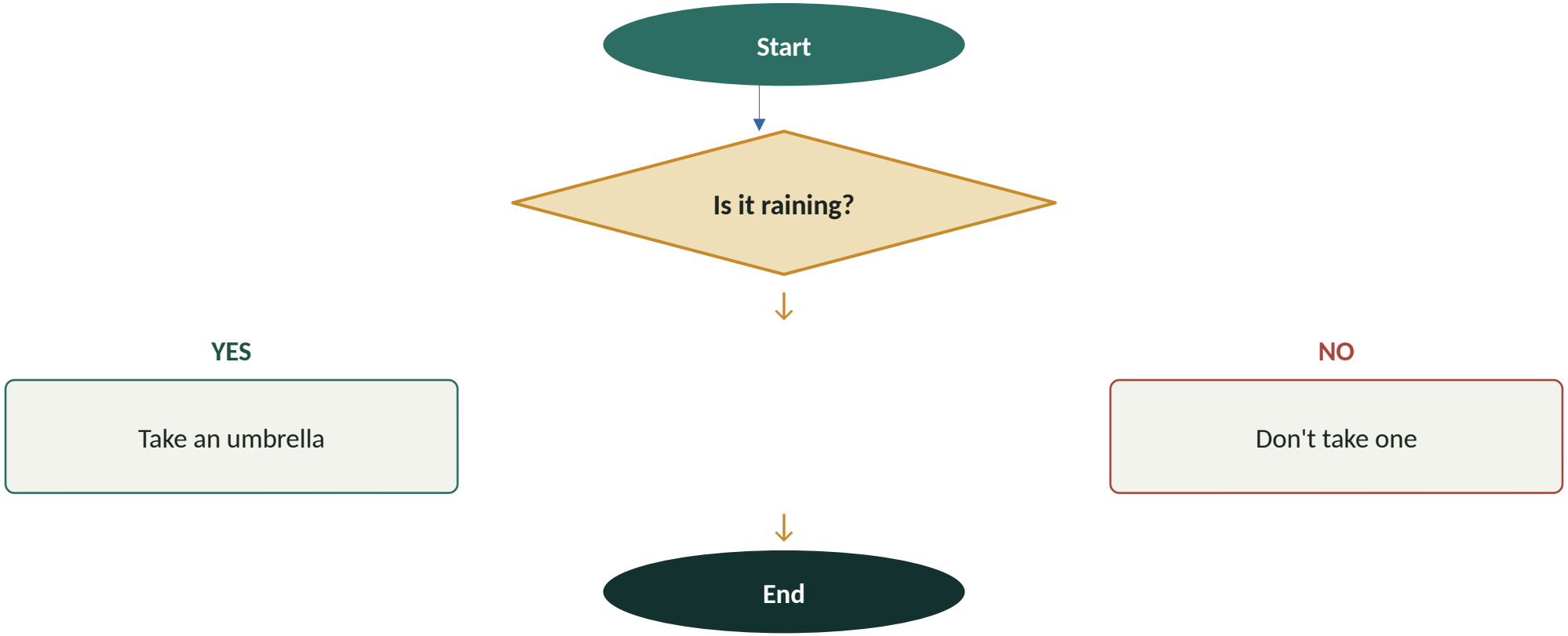
One step or action to perform



Diamond

A decision — always has a Yes and a No path

From Idea to Flowchart





The if Statement

- if checks a condition, and only runs the code below it when that condition is True
- The indented lines are what actually run — Python uses indentation instead of curly braces
- If the condition is False, Python simply skips that whole block

```
age = 20

if age >= 18:
    print("You can vote")
```



if / else

- else covers every other case — anything that didn't match the if
- Exactly one of the two blocks will always run. Never both, never neither.

```
age = 15

if age >= 18:
    print("You can vote")
else:
    print("Not old enough yet")
```



if / elif / else

- elif lets you check more conditions, one after another, in order
- Python checks each one top to bottom, and stops at the very first True it finds
- else still catches anything that matched none of the conditions above it

```
score = 82

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
else:
    print("Grade: F")
```

Nested if — An if Inside an if



A decision inside a decision

- A nested if is simply an if statement placed inside another if statement
- The inner if only even runs if the outer condition was already True
- Useful when a second decision only makes sense after the first has already been decided
- Analogy: first check if the door is unlocked — only then check if your key actually fits

Nested if — In Code

- Notice the extra indentation — the inner if sits one full level deeper than the outer one
- Python uses that indentation to know exactly which if belongs to which
- The inner if/else only gets checked at all when age >= 18 is True

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("Bring your ID next time")
else:
    print("Too young to enter")
```



Nested if vs. elif — Which One?



Use elif when...

- You're checking different values of the SAME thing
- Only ever one branch should run
- Example: turning a score into a single letter grade



Use Nested if when...

- The second check only matters if the first was already True
- You're combining two separate, related conditions
- Example: old enough AND has a valid ID

Conditional Mini Programs (1 & 2)

1. Even or Odd

```
number = int(input("Enter a number: "))

if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

2. Simple Login System

```
username = input("Username: ")
password = input("Password: ")

if username == "admin" and password == "1234":
    print("Login successful")
else:
    print("Login failed")
```

Mini Program: Ticket Price by Age

- Four age groups, four prices — a perfect fit for if / elif / else
- Each elif only needs to check one boundary, since anything caught above it never reaches this line
- Try tracing a few different ages through this by hand before running it

```
age = int(input("Enter your age: "))

if age < 5:
    price = 0
elif age < 18:
    price = 10
elif age < 60:
    price = 20
else:
    price = 12

print("Ticket price: $" + str(price))
```

Common Mistakes With if

Common Mistakes

- Using = instead of == (if age = 18 is an error)
- Forgetting the colon : at the end of the line
- Mixing tabs and spaces for indentation
- Writing "else if" instead of Python's elif

The Fix

- Always use == when comparing two values
- Every if, elif, and else line ends with a colon
- Use spaces only — most editors handle this automatically
- Remember: it's one word — elif

From Flowchart to Code — One Full Example

- Every diamond in a flowchart becomes an if, elif, or else
- Every rectangle becomes the code that runs inside that block
- Planning first — even briefly — is faster than debugging confused code later

```
score = int(input("Enter your score: "))

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print("Your grade is:", grade)
```

Bonus Challenge: Build a Calculator

- This one program uses almost everything from today — variables, operators, and conditionals
- Ask for two numbers and an operator, then use if/elif/else to decide what to compute
- Two branches below are left for you to finish — that's tonight's Task 10

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
operator = input("Choose an operator (+, -, *, /): ")

if operator == "+":
    print(num1 + num2)
elif operator == "-":
    print(num1 - num2)
elif operator == "*":
    # your turn: multiply num1 and num2
    pass
elif operator == "/":
    # your turn: divide num1 by num2
    pass
else:
    print("Not a valid operator")
```

Practice Set — Conditionals

Try these yourself first — no code shown, that's the point

1. Write an if/else that checks if a number is positive or negative
2. Write an if/elif/else that labels a temperature as Cold, Warm, or Hot
3. Use and inside an if to check discount eligibility (is_member and total_spent > 50)
4. Write an if/else that checks if a piece of text is empty ("" means empty)
5. Write a nested if: check age >= 18 first, then check has_ticket inside it



BONUS

A Few More Operators

Optional for today — good to know these exist.

Assignment Shortcut Operators



+=

x += 3 is short for x = x + 3



-=

x -= 3 is short for x = x - 3



***=**

x *= 3 is short for x = x * 3



/=

x /= 3 is short for x = x / 3

Increment & Decrement — Python's Twist

What Other Languages Do

- C, Java, and JavaScript have i++ and i--
- i++ means "increase i by 1"
- i-- means "decrease i by 1"
- Typing this in Python causes a SyntaxError

What Python Actually Does

- Python has no ++ or -- operators at all
- Use i += 1 instead of i++
- Use i -= 1 instead of i--
- It's one extra character to type — that's the whole trade-off

Increment & Decrement, in Real Python

- This is exactly the += and -= you just saw, used the way most people actually reach for them
- Try typing x++ yourself and read the error Python gives you — it's a good one to recognize early

```
x = 5

x += 1
print(x)  # 6

x -= 1
print(x)  # back to 5

# x++     <- would raise a SyntaxError in Python, try it!
```

Bitwise Operators

& AND

5 & 3 → 1

| OR

5 | 3 → 7

^ XOR

5 ^ 3 → 6

~ NOT

~5 → -6

<< Left Shift

5 << 1 → 10

>> Right Shift

5 >> 1 → 2

Identity & Membership Operators

- is checks whether two variables point to the exact same object — different from ==, which only checks if the values are equal
- in checks whether something exists inside a list, string, or similar collection
- Both read almost like plain English once you see them

```
fruits = ["apple", "banana", "mango"]

print("banana" in fruits)     # True
print("grape" not in fruits)  # True

a = None
print(a is None)            # True
```

Escape Sequences — Special Characters



A backslash with a special job

- Some characters are hard to type directly inside a string, like a new line or a quote mark
- An escape sequence starts with a backslash \ and tells Python "treat the next character specially"
- The two you'll use constantly: \n for a new line, and \t for a tab space

Common Escape Sequences



\n

Starts a new line



\t

Adds a tab space



****

A single backslash



\'

A single quote mark



\"

A double quote mark

Escape Sequences in Action

- Notice \n starts a new line right in the middle of one print() statement
- Without the backslash before a matching quote, Python would think the string ended too early

```
print("Line one\nLine two")
print("Name:\tAmeer")
print("She said \"hello\"")
print('It\'s a nice day')
print("Path: C:\\Users\\Ameer")
```

Practice Set — Bonus Operators

Try these yourself first — no code shown, that's the point

1. Use += to add 5 to a variable, then print it

2. Check if "cat" is in a list of animals using in

3. Try 6 & 3 and 6 | 3 — predict the answers before running them

4. Print your name and a hobby on two separate lines, using one print() and \n

Today's Tasks — Part A: Plan First

Small tasks — do them in order

1. Write pseudocode (plain English) for checking if a number is even or odd
2. Sketch a simple flowchart for that same idea, using today's 3 shapes
3. Try every arithmetic operator once in a script, and print each result
4. Use % to actually check if a number is even or odd, in real Python code

Today's Tasks — Part B: Operators & Conditionals

Small tasks — do them in order

- **5** Write one comparison operator expression and print the True/False result
- **6** Combine two conditions using and, then again using or — print both results
- **7** Write an if/else that checks if someone is old enough to vote (age >= 18)
- **8** Write an if/elif/else that assigns a letter grade from a score
- **9** Write a nested if: check age >= 18 first, then check has_ticket inside it
- **10** Turn your Task 1 pseudocode and Task 2 flowchart into real, working Python code
- **11** Build a simple calculator: ask for two numbers and an operator (+ - * /), then use if/elif/else to print the correct result

11 core tasks in total — due before our next class

Today's Tasks — Part C: Bonus (Optional)

Small tasks — do them in order

12 Use += to add 5 to a variable, then print the result

13 Try 6 & 3 and 6 | 3 in Python — predict the answers before running them

14 Check if "cat" is in a list of animals using in

Optional — try these only if Parts A & B are already done



RECAP

You can now make your program think. Every if is a fork in the road your code can take.

Next session: Lists and Tuples — Non-primitive Data Structures.